

device needs to be looked up. A serial port is controlled by the serial controller device, commonly known as an UART (Universal Asynchronous Receiver/Transmitter) or at times a USART (Universal Synchronous/Asynchronous Receiver/Transmitter). On PCs, the typical UART used is the PC16550D. The data-sheet for this, *uart_pc16550d.pdf*, has been included in this month's LFY CD, and also can be downloaded as part of the self-extracting package (<http://bit.ly/IOEUbJ>) used for the Linux device driver kit, available at <http://lddk.esrjan.com>.

Generally speaking, from where, and how, does one get these device data sheets? Typically, an online search with the corresponding device number should yield their data-sheet links. Then, how does one get the device number? Simple...by having a look at the device. If it is inside a desktop, open it up and check it out. Yes, this is the least you may have to do to get going with the hardware, in order to write device drivers. Assuming all this has been done, it is time to peep into the data sheet of the PC16550D UART.

Device driver writers need to understand the details of the registers of the device, as it is these registers that writers need to program, to use the device. Page 14 of the data sheet (also shown in Figure 2) shows the complete table of all the twelve 8-bit registers present in the UART PC16550D. Each of the eight rows corresponds to the respective bit of the registers. Also, note that the register addresses start from 0 and goes up to 7. The interesting thing about this is that a data sheet always gives the register offsets, which then needs to be added to the base address of the device, to get the actual register addresses. Who decides the base address and where is it obtained from? Base addresses are typically board/platform specific, unless they are dynamically configurable like in the case of PCI devices. In this case, i.e., a serial device on x86, it is dictated by the x86 architecture—and that precisely was the starting serial port address mentioned above—0x3F8. Thus, the eight register offsets, 0 to 7, exactly map to the eight port addresses 0x3F8 to 0x3FF. So, these are the actual addresses to be read or written, for reading or writing the corresponding serial registers, to achieve the desired serial operations, as per the register descriptions.

All the serial register offsets and the register bit masks are defined in the header `<linux/serial_reg.h>`. So, rather than hard-coding these values from the data sheet, the corresponding macros could be used instead. All the following code uses these macros, along with the following:

```
#define SERIAL_PORT_BASE 0x3F8
```

Operating on the device registers

To summarise the decoding of the PC16550D UART data sheet, here are a few examples of how to do read and write operations of the serial registers and their bits.

Reading and writing the 'Line Control Register (LCR)':

```
u8 val;

val = inb(SERIAL_PORT_BASE + UART_LCR /* 3 */);
outb(val, SERIAL_PORT_BASE + UART_LCR /* 3 */);
```

Setting and clearing the 'Divisor Latch Access Bit (DLAB)' in LCR:

```
u8 val;

val = inb(SERIAL_PORT_BASE + UART_LCR /* 3 */);

/* Setting DLAB */
val |= UART_LCR_DLAB /* @x08 */;
outb(val, SERIAL_PORT_BASE + UART_LCR /* 3 */);

/* Clearing DLAB */
val &= ~UART_LCR_DLAB /* @x08 */;
outb(val, SERIAL_PORT_BASE + UART_LCR /* 3 */);
```

Reading and writing the 'Divisor Latch':

```
u8 dlab;
u16 val;

dlab = inb(SERIAL_PORT_BASE + UART_LCR);
dlab |= UART_LCR_DLAB; // Setting DLAB to access Divisor Latch
outb(dlab, SERIAL_PORT_BASE + UART_LCR);
```

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. info@technoinfotech.com